

COPILOTTRAINING
TECHNICAL BRIEFING

Unattended Agents

Loops That Run Without You and Stop on Their Own Terms

Runs on its own. Stops on evidence. Ends with your review.

Agentic Systems · 2026

🤔 THE CORE QUESTION

"What lets an agent run without you — and stop when it has to?"

Single-shot agents wait for the next prompt.

Unattended agents verify their own progress, stop at their limits, and hand you the result.

Platform Engineer

Designing shared agent pipelines that run unattended across teams

Senior Developer

Moving beyond one-shot Copilot into verified autonomous workflows

Team Lead

Governing agent cost, safety, and reliability at scale

Loops, not prompts

Claude Code lead Boris Cherny now writes the loops that prompt Claude, as reported by The New Stack

18 min alert to draft PR

Illustrative design: scan → fix → critic → draft PR, with one human review at the end

Most agentic failures

Trace to missing circuit breakers and watchdogs — the controls teams skip first

📖 TABLE OF CONTENTS

PART 1

The Loop

Definition + Trigger

What an unattended agent is and what fires it more than once

PART 2

Multi-Agent Design

Handoff + Topology

How agents coordinate state, escalate, and organize

PART 3

Production Controls

Don't skip these

Circuit breaker, bounded execution, and goal-watchdog

PART 4

Wiring It Together

Alert to draft PR

Full multi-agent pipeline from scheduled alert check to reviewed PR

PART 1

The Loop

What IS an unattended agent — and what triggers it to run more than once?

Definition

Act → verify → repeat until goal met — then stop

Trigger

Kick off by hand or with /every in Copilot CLI

Simple Path

Three agent files + shared state, no framework

```
loop-engineering
↳ act → verify → repeat until goal met
```

What Is an Unattended Agent?

One sentence. Everything else in this talk follows from it.

Loop

An agent that continues to act and verify in a cycle until the goal is demonstrably met — then stops.

Triggered once

One prompt or a /every schedule starts the loop — no prompting between iterations

Runs until verified

The agent reasons, acts, and checks its own goal condition — not just assumes it's done

Stops on proof

The loop terminates when the goal is demonstrably true, or when controls say stop

Not orchestration

Orchestration runs once. An unattended agent runs until the work is verifiably complete.

The hardest engineering challenge is not starting the loop — it is stopping it correctly.

One Kickoff, Then /every Keeps It Looping

Copilot CLI session

```
$ copilot --experimental

# one pass, on demand:
> Use gh to review all open issues.
  Apply 'stale' to any with no activity
  for 3 days. Close issues stale 7+ days
  with a polite note.

# the same pass, every day:
> /every 1d Use gh to review all open
  issues ... (same prompt)

# review or remove schedules:
> /every
```

▶️ Kick off by hand

One prompt to the orchestrator runs a pass — the fastest way to watch the loop work

⏰ Repeat with /every

/every 1h repeats the prompt on an interval; /after 30m schedules one later run (experimental)

💻 Where the loop runs

In your CLI session, with your credentials and working tree — the schedule lasts as long as the session

The Orchestrator — Loop Logic in Plain Instructions

.github/agents/loop-orchestrator.agent.md

```
---
description: Loop orchestrator
tools: [read, edit, search,
        agent, github/*]
---
1. Read `.github/loop-state.json`.
2. If iteration_count ≥ max_iterations,
   write an escalation note and stop.
3. Check all goal.conditions - if all
   are true, write a summary and stop.
4. For each task in state.pending,
   invoke the matching worker
   custom agent (agent tool).
5. Update state.results and increment
   iteration_count.
6. Repeat from step 2.
```

Trigger (step 1)

A kickoff prompt or /every 1h in Copilot CLI runs this file as the session's agent

Goal + Verify (steps 2–3)

Checks verifiable conditions before delegating — stops on proof, not assumption

Execute + Control (steps 4–6)

Delegates to workers, updates state, and caps itself at max_iterations

The Worker — Accepts a Task, Returns a Result

.github/agents/loop-reviewer.agent.md

```
---
description: Code review worker
tools: [read, edit, search,
        github/*]
---
1. Read your task from
   .github/loop-state.json
   (tasks.pending[0]).
2. Perform the code review
   for the assigned PR.
3. Write findings back to
   state.results.review.
4. Set tasks.complete[] and
   remove from tasks.pending.
5. Update goal.conditions
   [review_approved] to true
   if review passes.
```

📥 Reads task from shared state

The state file is the handoff — no direct agent-to-agent call needed

📤 Writes result back to state

Orchestrator reads results on next iteration and decides what to do next

✅ Updates goal conditions

Workers own their piece of the goal — the orchestrator just checks the total

loop-state.json — The Shared Brain

.github/loop-state.json

```
{
  "iteration_count": 2,
  "max_iterations": 5,
  "goal": {
    "conditions": [
      { "name": "review_approved",
        "met": true },
      { "name": "tests_passing",
        "met": false }
    ]
  },
  "tasks": {
    "pending": ["test_run"],
    "complete": ["code_review"]
  },
  "results": {
    "review": "No critical issues"
  }
}
```

📖 All agents read from here

The orchestrator, reviewer, and tester all read the same file — no direct coupling

🖍️ All agents write back

Any writable surface works. A repo file changes on the PR branch, so run one loop at a time

🔍 State is the audit trail

iteration_count, results, and goal.conditions give full provenance without extra tooling

PART 2

Multi-Agent Design

How agents hand off work, track state, and know when to escalate

Structured Handoff

State entries with intent, constraints, escalate_on

Lifecycle States

Idle → Suspended: where loops silently fail

Topology

Flat-pool, hierarchical, or pipeline — your choice

```
multi-agent-design
↳ handoff → escalate → verify → next agent
```

Two Ways to Pass Work Between Agents

Simple Path

Write to loop-state.json
tasks.pending, goal.conditions, results

Worker reads its task
No direct call — state is the handoff

Key fields to include
intent · constraints · escalate_on

Structured Path

Typed task envelope (JSON)
task_id · trace_id · sender · receiver

escalate_on conditions
explicit triggers that return control upward

on_complete routing
declarative: who gets the baton next

```
{ "intent": "review_pr",
  "escalate_on": ["deadline_exceeded"],
  "on_complete": { "trigger": "tester" } }
```

Agent Lifecycle — The State Machine Every Loop Needs

Most silent failures live in the Idle → Suspended gap

❌ Healthy Execution

1. **Created**
   Task received from orchestrator
2. **Initialized**
   Context loaded, tools verified available
3. **Active → Terminated**
   Goal verified; audit trail written
4. **Result in state store**
   Orchestrator proceeds to next task

✅ Loop continues or terminates cleanly

✅ Failure Path

1. **Created**
   Task received from orchestrator
2. **Active**
   Agent begins ReAct cycle
3. **Idle**
   Waiting for a tool — the lock stays held while it waits
4. **Suspended**
   A limit fires — iteration cap, credit cap, or the next pass's stale-lock check

🔌 Circuit breaker required — covered in Section 3

Idle → Suspended: where most production loop failures begin

Three Topologies — Pick One Before Writing Any Agent



Flat Pool

One orchestrator dispatches to N identical workers; results aggregated. Use for parallelizable homogeneous tasks.

- **.github/agents/ = orchestrator + N workers**
- **Aggregate with majority-vote or first-success**



Hierarchical

Orchestrator → specialized workers → critic. Use when tasks need specialization and a quality gate.

- **Workers have distinct capabilities**
- **Critic validates before accepting output**
- **Recommended default for engineering pipelines**



Pipeline

Each stage strictly depends on previous output. Use when the workflow is sequential by nature.

- **analyze → implement → test → review**
- **Stage failure triggers rollback or escalate**

Four Fields That Make Handoff Traceable

🔎

trace_id

Links every agent action in this session — makes post-hoc debugging possible without extra tooling

🎯

intent

What the receiving agent is being asked to do — not how, just what. Keeps agents swappable.

⚠️

escalate_on

Explicit conditions that return control to orchestrator or human — never implicit, never ad hoc

➡️

on_complete

Who receives the baton when this agent finishes — makes the pipeline declarative rather than imperative

💡 These four fields work whether the handoff is a JSON file, a state store entry, or a structured task envelope.

PART 3

Production Controls

The three controls most teams skip and most failures trace back to

🔌

Circuit Breaker

Iteration, time, and rejection limits the orchestrator counts

💰

Bounded Execution

A credit cap and denied tools the CLI enforces

👁️

Goal-Watchdog

The next pass escalates a run that stopped part-way

```
production-controls
↳ most failures trace here — catch them before deploy
```

Three Controls, Three Enforcers



Circuit Breaker

Enforced by the orchestrator. It counts in loop-state.json and escalates at each limit — reliable when it counts correctly, and history shows whether it did.

- **max_iterations: 8**
- **timeout_minutes: 45**
- **critic.max_rejections: 2**



Bounded Execution

Enforced by Copilot CLI. These hold whatever the agent intends. The cap counts AI credits for the objective — no surface caps tokens per agent.

- **/goal … --max-ai-credits 5 pauses spend**
- **--deny-tool blocks merge and review**
- **Esc stops a pass on the spot**



Goal-Watchdog

Enforced by the next pass. Each /every pass reads the lock first; a run that stopped part-way escalates instead of resuming.

- **Lock older than 45 min → stale_run**
- **Escalates to the security team**
- **Never resumes a half-changed worktree**

PART 4

Wiring It Together

An illustrative design: from a scheduled alert check to a reviewed draft PR.

⏱️

The Outcome

Runtime table: T+0 to T+18, row by row

🗂️

Agent Files

Who participates and how they share state

🔍

Audit Trail

Full provenance via trace_id, trigger to termination

```
wiring-it-together
↳ illustrative · ~18 minutes · human merges
```

18 Minutes to a Reviewable PR. One Human Decision.

Illustrative timeline: critical alert → CVE fix → draft PR passing CI

```
$ /every 1h pass: new Dependabot alert [CRITICAL] — CVE in dependency
🔄 Copilot CLI runs the pass as loop-orchestrator
✅ T+1 min: Orchestrator dispatches a task envelope to vuln-scan-agent
✅ T+3 min: Scan agent returns affected_files_report
✅ T+7 min: Fix-proposal agent returns proposed_diff
─────────────────────────────────────────────────────────────────
✅ T+10 min: Test-runner validates fix against affected test suite
✅ T+11 min: Security-critic approves (confidence 0.91 > 0.85 threshold)
✅ T+12 min: PR-author opens draft PR with label [security, automated]
─────────────────────────────────────────────────────────────────
✅ T+18 min: CI passes. All 4 goal conditions true. Loop stops; review requested.
  Audit trail written: trace_id · per-dispatch history · decision chain
```

Illustrative timings · 0 prompts from trigger to draft PR · security-team merges

Five Agent Files. One Shared State. No Framework.

.github/agents/

loop-orchestrator.agent.md
Reads state, dispatches workers, checks goals

vuln-scan-agent.agent.md
Finds affected files → affected_files_report

fix-proposal-agent.agent.md
Generates the fix → proposed_diff

security-critic-agent.agent.md
Checks fix and tests → approve or reject

pr-author-agent.agent.md
Opens the draft PR after critic approval

What Makes It Work

Scheduled trigger
One kickoff by hand, or /every 1h in Copilot CLI

loop-state.json
Shared by all five agents — no direct coupling

Goal conditions
pr_opened · ci_passing · cve_resolved · no_new_cves

Circuit breaker
max_iterations: 8 · timeout: 45 min · one run at a time

```
# Copy-ready starter:
# tech-talks/unattended-agents/
#   examples/loop-starter/
```

THE VALUE RECAP

From One-Shot Prompts to Unattended Agents

Before

- Hand-prompt every step; agent stops when it thinks it's done
- No recurring trigger — workflows run once and wait for a human to restart
- Ad hoc messages between agents — no trace ID, no escalation path
- No iteration cap — a stalled tool call runs until manual cancellation

After

- Trigger once; agent loops until goal condition is verifiably true
- /every in Copilot CLI repeats the pass on an interval — no hand-prompting between runs
- Typed state entries with trace ID, constraints, and escalate_on fields
- Agent-counted limits, a CLI credit cap, and a next-pass stall check each stop the loop

4
verifiable goal conditions checked before the loop stops

18 min
alert to reviewable draft PR in the illustrative design

1
human decision: the merge stays with the security team

✅ WHAT YOU CAN DO TODAY

Today

- Start copilot --experimental and run /every 1d with a simple triage prompt — minimum viable loop trigger, no code required
- Write one recurring task as a verifiable goal condition — not a description, a checkable criterion

This Week

- Wire a three-agent pipeline (orchestrator + worker + critic) with a shared loop-state.json
- Run one pass of the loop starter by hand and read loop-state.json evidence before adding /every

This Month

- Give every scheduled loop all three enforcers: counted limits, a --max-ai-credits cap, and a stale-lock check
- Have the orchestrator write the evidence checklist into every draft PR body

🔑 Key Takeaway

Every unattended agent starts with a scheduled trigger, a verifiable goal, and a stop condition.

REFERENCES

Official Documentation

Copilot CLI command reference
/every and /after scheduling, --agent, --allow-tool and --deny-tool

Custom agents configuration
Tool aliases (read, edit, search, agent) for orchestrator and worker agent files

Open Source & Guides

cobusgreyling/loop-engineering
Loop patterns: daily triage, PR babysitter, CI sweeper, dependency sweeper

gim-home/herald
Agent management harness: lifecycle state machine, coordination primitives

The Art of Loop Engineering — LangChain Blog
Custom termination logic, dynamic replanning, nested loops, and state management

Agentic Loops: From ReAct to Loop Engineering (2026 Guide)
Loop anatomy, pattern taxonomy, and why loop engineering supersedes prompt engineering



Unattended Agents

AUTONOMOUS BY DESIGN. VERIFIABLE BY DEFAULT.

The Definition

An unattended agent continues to act and verify until the goal is demonstrably met — then stops.

The Trigger

One kickoff by hand, or /every 1h in Copilot CLI. No infrastructure required.

The Design

Critical alert to a reviewable draft PR in about 18 minutes (illustrative), with one human merge.

What recurring task on your team could run as an unattended agent starting tomorrow?